



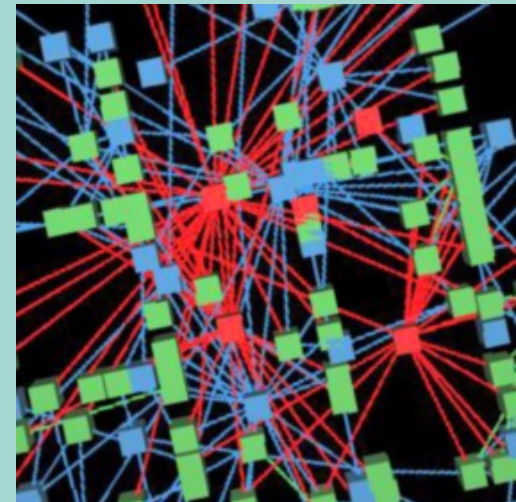
University
of Basel

Chapter 3.4:

Resilience and WAL

Foundations of Distributed Systems

F. Ciorba, H. Schuldt, C. Tschudin, I. Wagner



Overview of Chapter 3.4 - Resilience and WAL

1.1 Journaling File System (showing the problem)

1.2 WAL: Write-Ahead-Log intro (slides Seshan, CMU)

1.3 WAL: detailed example (slides Peter Du, Anderson)

1.4 Atomicity for rename () in the UNIX file system

Journaling File Systems

Questions answered in this lecture:

Why is it hard to maintain on-disk consistency?

How does the FSCK tool help with consistency?

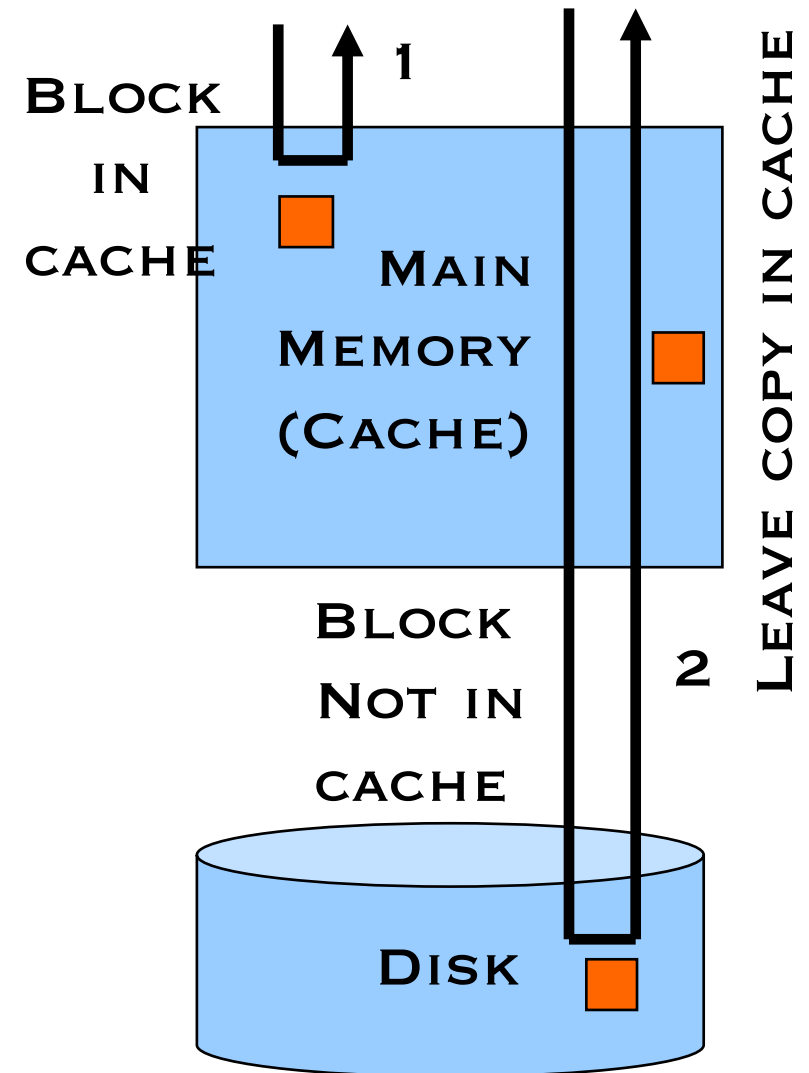
What information is written to a journal?

What 3 journaling modes does Linux ext3 support?

Review: The I/O Path (Reads)

Read() from file

- Check if block is in cache
- If so, return block to user [1 in figure]
- If not, read from disk, insert into cache, return to user [2]



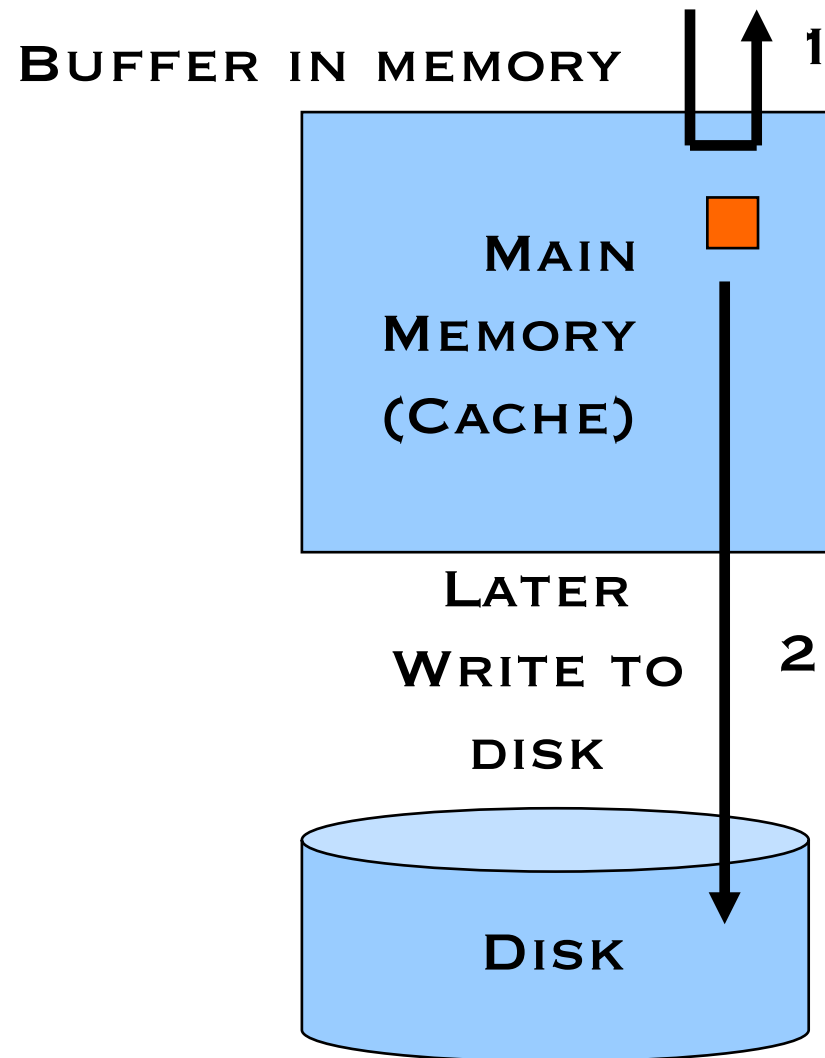
Review: The I/O Path (Writes)

Write() to file

- Write is buffered in memory ("write behind") [1]
- Sometime later, OS decides to write to disk [2]

Why delay writes?

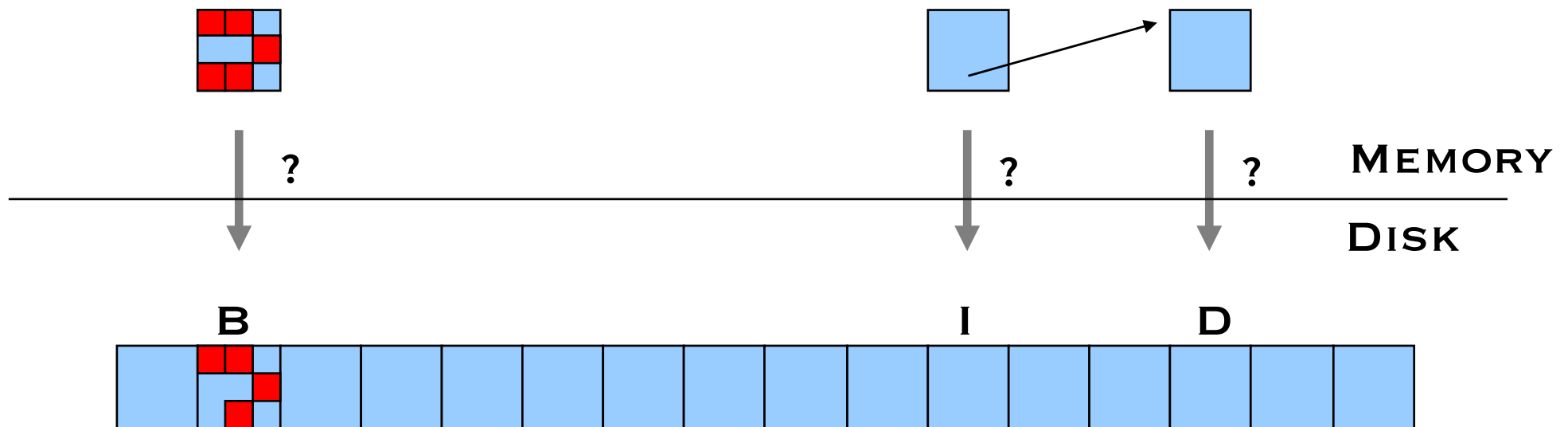
- Implications for performance
- Implications for reliability



Many “dirty” blocks in memory: What order to write to disk?

Example: Appending a new block to existing file

- Write data bitmap B (for new data block),
write inode I of file (to add new pointer, update time),
write new data block D



The Problem

Writes: Have to update disk with N writes

- Disk does only a single write atomically

Crashes: System may crash at arbitrary point

- Bad case: In the middle of an update sequence

Desire: To update on-disk structures **atomically**

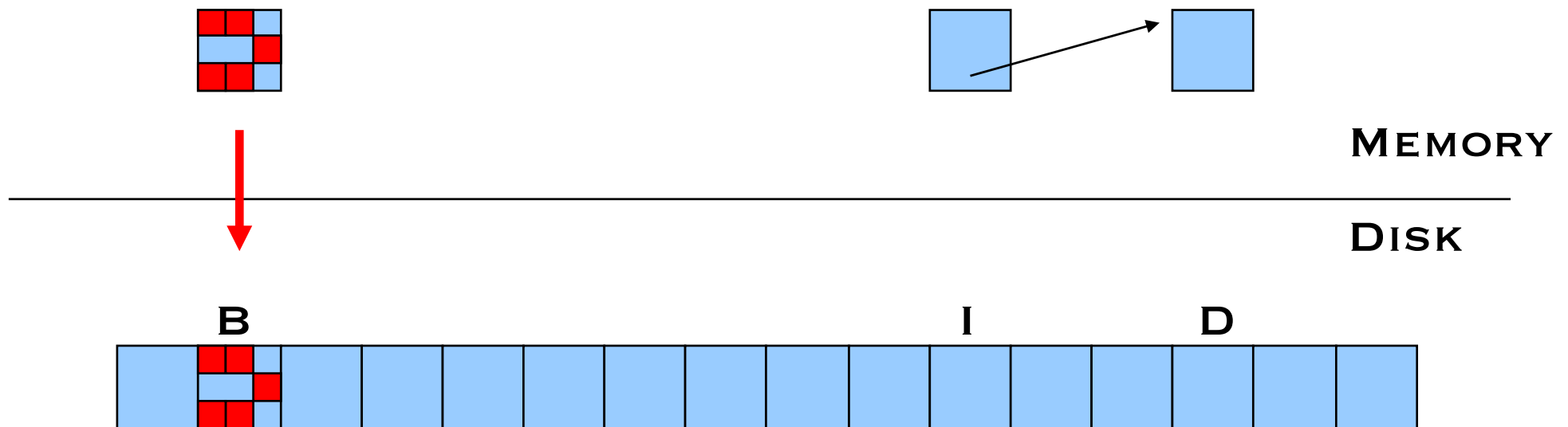
- Either all should happen or none

Example: Bitmap first

Write Ordering: Bitmap (B), Inode (I), Data (D)

- But CRASH after B has reached disk, before I or D

Result?

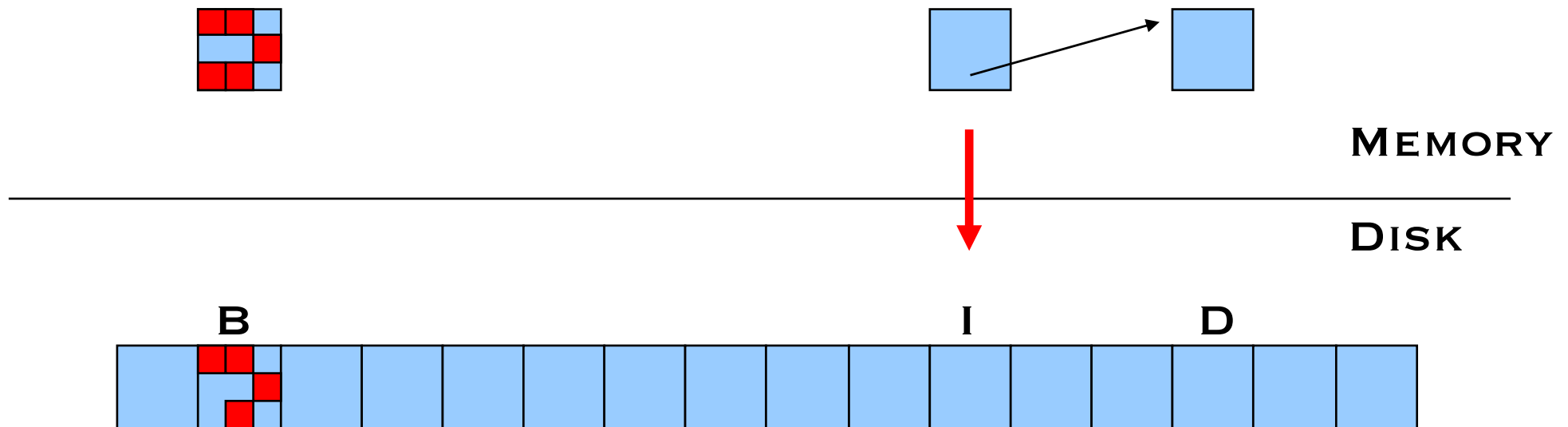


Example: Inode first

Write Ordering: Inode (I), Bitmap (B), Data (D)

- But CRASH after I has reached disk, before B or D

Result?

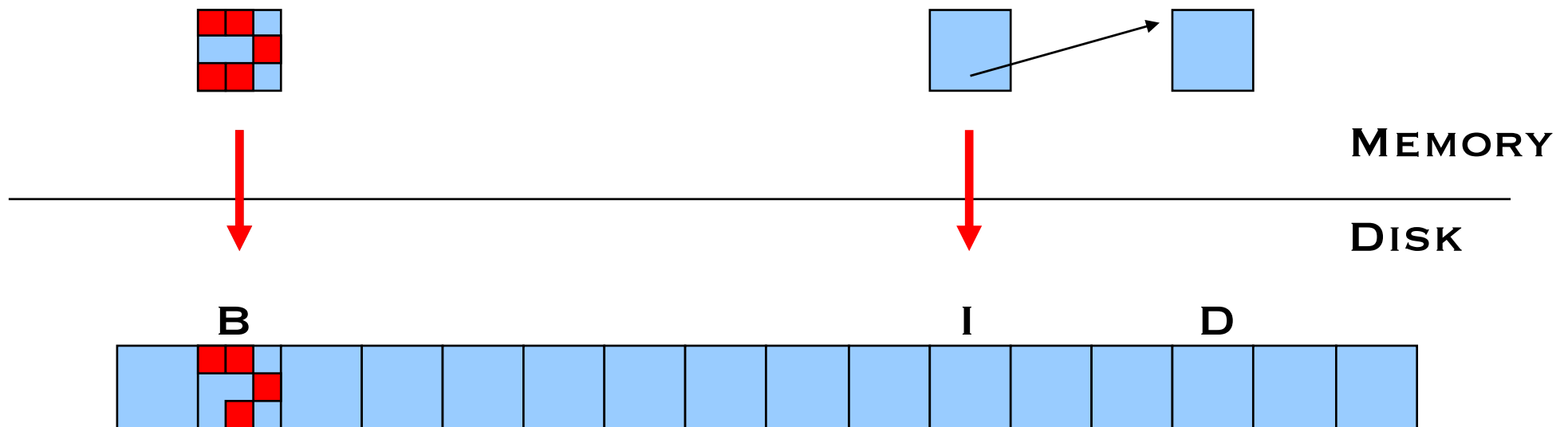


Example: Inode first

Write Ordering: Inode (I), Bitmap (B), Data (D)

- CRASH after I AND B have reached disk, before D

Result?

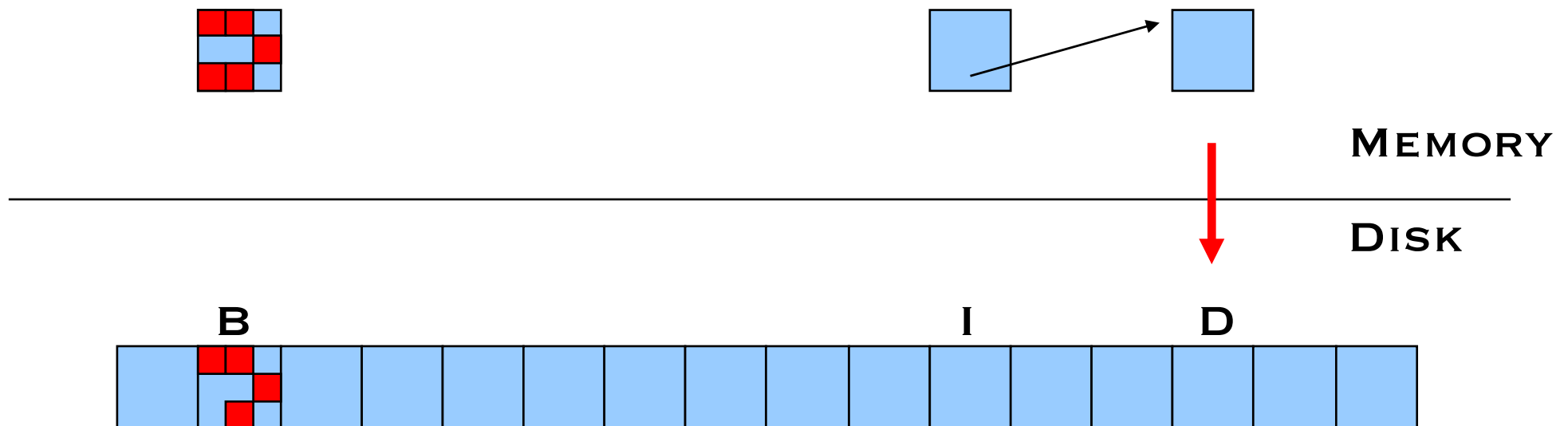


Example: Data first

Write Ordering: Data (D) , Bitmap (B), Inode (I)

- CRASH after D has reached disk, before I or B

Result?



Traditional Solution: FSCK

FSCK: “file system checker”

When system boots:

- Make multiple passes over file system, looking for inconsistencies
 - e.g., inode pointers and bitmaps, directory entries and inode reference counts
- Either fix automatically or punt to admin
- Does fsck have to run upon every reboot?

Main problem with fsck: **Performance**

- Sometimes takes hours to run on large disk volumes

How To Avoid The Long Scan?

Idea: Write something down to disk before updating its data structures

- Called the “write ahead log” or “journal”

When crash occurs, look through log and see what was going on

- Use contents of log to fix file system structures
- The process is called “recovery”

Transactions in ext3 Data Journaling Mode

Same example: Update Inode (I), Bitmap (B), Data (D)

First, write to journal:

- Transaction begin (Tx begin)
- Transaction descriptor (info about this Tx)
- I, B, and D blocks (in this example)
- Transaction end (Tx end)

Then, “checkpoint” data to fixed ext2 structures

- Copy I, B, and D to their fixed file system locations

Finally, free Tx in journal

- Journal is fixed-sized circular buffer, entries must be periodically freed

Overview of Chapter 3.4 - Resilience and WAL

1.1 Journaling File System (showing the problem)

1.2 WAL: Write-Ahead-Log intro (slides Seshan, CMU)

1.3 WAL: detailed example (slides Peter Du, Anderson)

1.4 Atomicity for rename () in the UNIX file system

Committing Distributed Transactions

Transactions may touch data stored at more than one site.

Each site commits (i.e., logs) its updates independently.

Problem: any site may fail while a commit is in progress, but after updates have been logged at another site.

An action could “partly commit”, violating atomicity.

Basic problem: individual sites cannot unilaterally choose to abort without notifying other sites.

“Log locally, commit globally.”

Logging

Key idea: supplement the home data image with a *log* of recent updates and/or events.

append-only

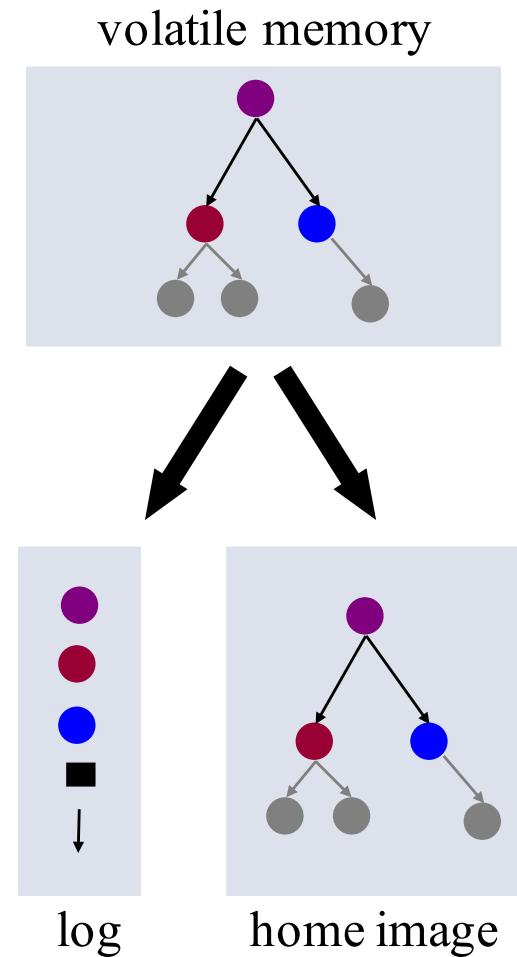
sequential access (faster)

preserves order of log entries

enables *atomic commit* with a single write

Recover by traversing, e.g., “replaying”, the log.

Logging is fundamental to database systems and other storage systems.



Achieving Atomic Durability

Atomic durability dictates that the system schedule its stable writes in a way that guarantees two key properties:

1. *Each transaction's updates are **tentative** until commit.*

Database state must not be corrupted with uncommitted updates.

If uncommitted updates can be written to the database, it must be possible to **undo** them if the transaction fails to commit.

2. *Buffered updates are written to stable storage synchronously with commit.*

Option 1: **force** dirty data out to the permanent (*home*) database image at commit time.

Option 2: commit by recording updates in a *log* on stable storage, and defer writes of modified data to home (**no-force**).

Atomic Durability with *Force*

A *force* strategy writes all updates to the home database file on each commit.

- must be synchronous
- disks are block-oriented devices

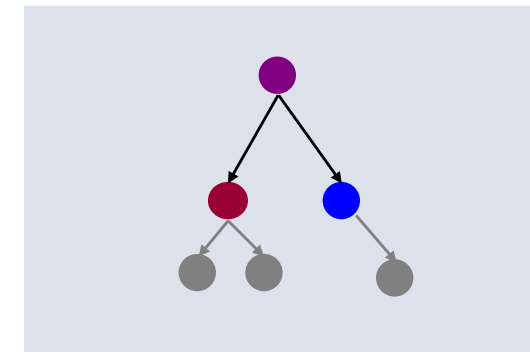
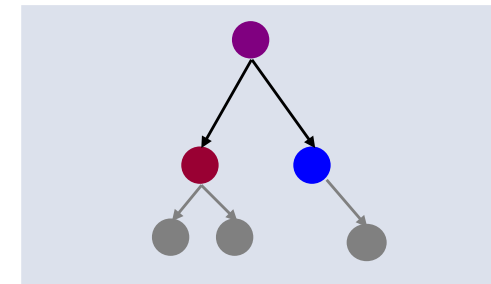
What if items modified by two different transactions live on the same block?

need page/block granularity locking

- writes may be scattered across file
- poor performance

What if the system fails in the middle of the stream of writes?

volatile memory

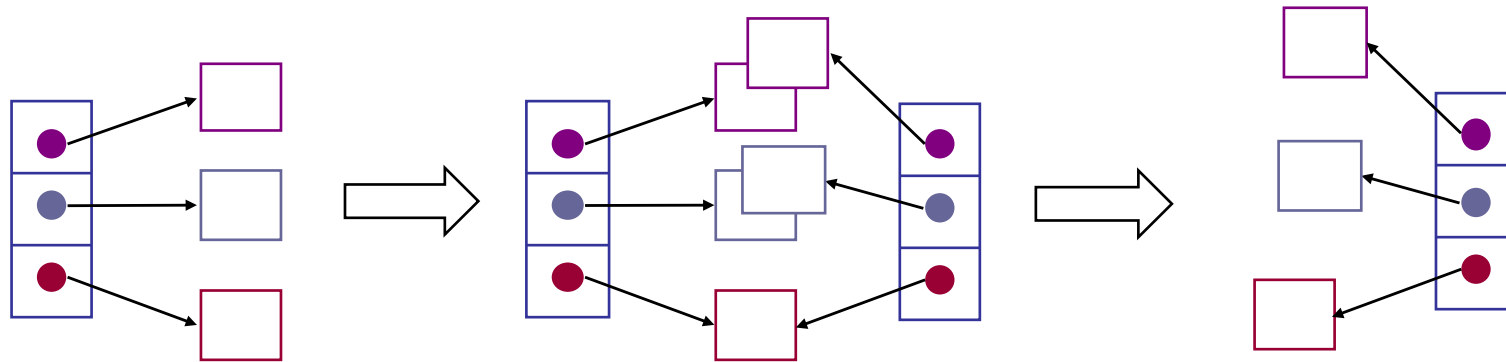


stable storage (*home*)

Shadowing

Shadowing is the basic technique for doing an *atomic force*.

reminiscent of *copy-on-write*



1. starting point
modify purple/grey blocks

2. write new blocks to disk
prepare new block map

3. overwrite block map
(*atomic commit*)
and free old blocks

Frequent problems: nonsequential disk writes, damages
clustered allocation on disk.

No-Force Durability with Logging

Logging appends updates to a sequential file in temporal order.

- **Durability**

The log supplements but does not replace the home image; to recover, replay the log into the saved home image.

The home image may be optimized for reads since there is no need to force updates to home on transaction commit.

- **Atomicity**

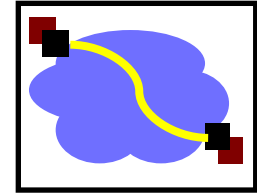
Key idea: terminate each group of updates with a *commit record* (including transaction ID) written to the log tail *atomically*.

- **Performance**

The log localizes updates that must be done synchronously, and so is well-suited to rotational devices with high seek times.

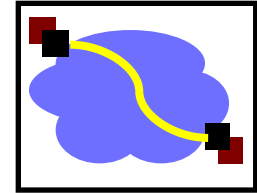
Drawback: some updates are written to disk twice (log and home).

Shadow Paging Vs WAL



- Shadow Pages
 - Provide Atomicity and Durability, “page” = unit of storage
 - Idea: When writing a page, make a “shadow” copy
 - No references from other pages, edit easily!
 - ABORT: discard shadow page
 - COMMIT: Make shadow page “real”. Update pointers to data on this page from other pages (recursive). Can be done atomically
 - Essentially “copy-on-write” to avoid in-place page update

Shadow Paging vs WAL



- Write-Ahead-Logging
 - Provide Atomicity and Durability
 - Idea: create a log recording every update to database
 - Updates considered reliable when stored on disk
 - Updated versions are kept in memory (page cache)
 - Logs typically store both REDO and UNDO operations
 - After a crash, recover by replaying log entries to reconstruct correct state
- WAL is more common, fewer disk operations, transactions considered committed once log written.

Overview of Chapter 3.4 - Resilience and WAL

1.1 Journaling File System (showing the problem)

1.2 WAL: Write-Ahead-Log intro (slides Seshan, CMU)

1.3 WAL: detailed example (slides Peter Du, Anderson)

1.4 Atomicity for rename () in the UNIX file system

Write-Ahead Logging

- In addition to evolving the state in RAM and on disk, keep a separate, **on-disk log** of all operations
 - Transaction **begin**, **commit**, **abort**
 - All **updates** (e.g., $X = X - \$20$; $Y = Y + \$20$)
- A transaction's operations are **provisional** until “commit” outcome is logged to disk
 - The result of these operations **will not be revealed** to other clients in meantime (i.e., new value of X will only be revealed after transaction is committed)
- Observation:
 - Disk writes of single pages/blocks are atomic, but disk writes across pages may not be

begin/commit/abort records

- Log Sequence Number (LSN)
 - Usually implicit, the address of the first-byte of the log entry
- LSN of previous record for transaction
 - Linked list of log records for each transaction
- Transaction ID
- Operation type

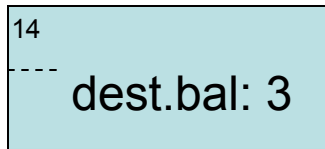
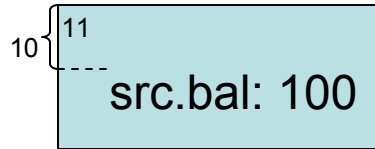
update records

- Need all information to undo and redo the update
 - prevLSN + xID + opType as before
 - The update itself, e.g.:
 - the update location (usually pageID, offset, length)
 - old-value
 - new-value

```
xId = begin();    // suppose xId <- 42    Log:  
src.bal -= 20;  
dest.bal += 20;  
commit(xId);
```

Disk:

Page cache:



Transaction table:

Dirty page table:

```

→ xId = begin();    // suppose xId <- 42
  src.bal -= 20;
  dest.bal += 20;
  commit(xId);

```

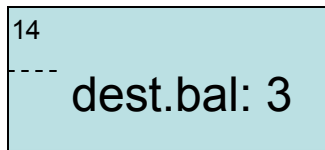
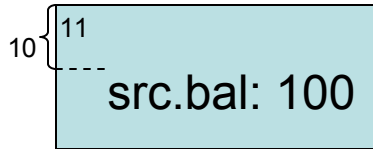
Log:

780

prevLSN:	0
xId:	42
type:	begin

Disk:

Page cache:



Transaction table:

42: prevLSN = 780

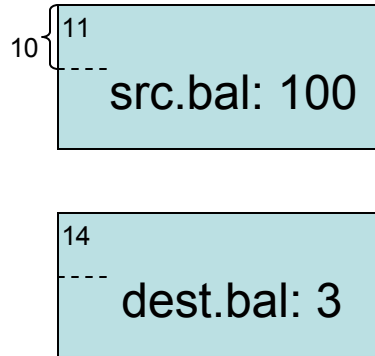
Dirty page table:

```

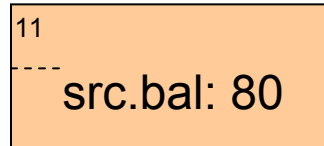
xId = begin();    // suppose xId <- 42
→ src.bal -= 20;
  dest.bal += 20;
  commit(xId);

```

Disk:



Page cache:



Log:

780	prevLSN: 0	
	xId: 42	
	type: begin	
	⋮	
860	prevLSN: 780	
	xId: 42	
	type: update	
	page: 11	} src.bal
	offset: 10	
	length: 4	
	old-val: 100	
	new-val: 80	

Transaction table:

42: prevLSN = 860

Dirty page table:

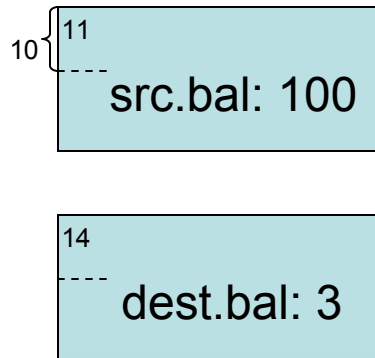
11: firstLSN = 860, lastLSN = 860

```

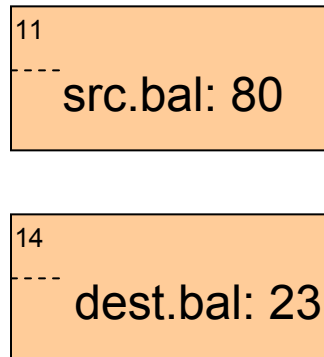
xId = begin();    // suppose xId <- 42
src.bal -= 20;
→ dest.bal += 20;
commit(xId);

```

Disk:



Page cache:



Transaction table:

42: prevLSN = 902

Dirty page table:

11: firstLSN = 860, lastLSN = 860

14: firstLSN = 902, lastLSN = 902

Log:

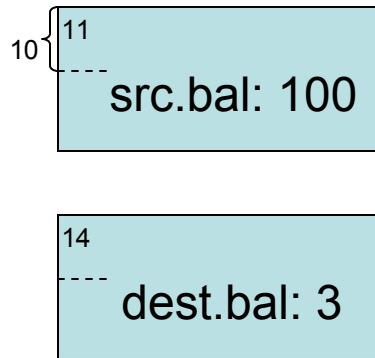
780	prevLSN: 0 xId: 42 type: begin
	⋮
860	prevLSN: 780 xId: 42 type: update page: 11 offset: 10 length: 4 old-val: 100 new-val: 80
	⋮
902	prevLSN: 860 xId: 42 type: update page: 14 offset: 10 length: 4 old-val: 3 new-val: 23

```

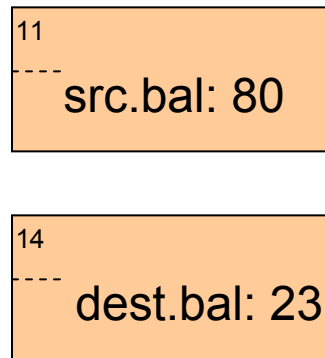
xId = begin();    // suppose xId <- 42
src.bal -= 20;
dest.bal += 20;
→ commit(xId);

```

Disk:



Page cache:



Transaction table:

Dirty page table:

11: firstLSN = 860, lastLSN = 860
 14: firstLSN = 902, lastLSN = 902

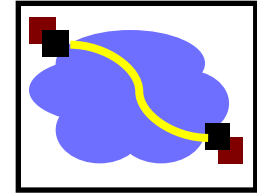
non-log pages may remain in memory

must flush the log to disk!

Log:

780	prevLSN: 0 xId: 42 type: begin
	⋮
860	prevLSN: 780 xId: 42 type: update page: 11 offset: 10 length: 4 old-val: 100 new-val: 80
	⋮
902	prevLSN: 860 xId: 42 type: update page: 14 offset: 10 length: 4 old-val: 3 new-val: 23
	⋮
960	prevLSN: 902 xId: 42 type: commit

Recovery using WAL – 3 passes



- Analysis Pass
 - Reconstruct TT and DPT (from start or last checkpoint)
 - Get copies of all pages at the start
- Recovery Pass (redo pass)
 - Replay log forward, make updates to all dirty pages
 - Bring everything to a state at the time of the crash
- Undo Pass
 - Replay log file backward, revert any changes made by transactions that had not committed (use PrevLSN)
 - For each write Compensation Log Record (CLR)
 - Once you reach BEGIN TXN, write an END TXN entry

Overview of Chapter 3.4 - Resilience and WAL

1.1 Journaling File System (showing the problem)

1.2 WAL: Write-Ahead-Log intro (slides Seshan, CMU)

1.3 WAL: detailed example (slides Peter Du, Anderson)

1.4 Atomicity for rename () in the UNIX file system

Atomicity for `rename()` in the UNIX file system

Problem statement:

- existing file, we have to process an incoming event
- file to be updated at *several* places (e.g. `seek(...); write(...)`)
- such that *all changes are applied*, or none of them

In other words:

crash-resilience: atomic update of multiple file locations, in user space

Approach:

- make a copy of the file
- update the copy (at multiple places)
- call `rename()` to replace the old file with the new one

(corresponds to “force durability”, from section 1.2, using a shadow file)

Atomicity for `rename()` in the UNIX file system (contd)

```
rename(char *old, char *new);           // part of POSIX API
```

From the man page (MacOS):

The `rename()` system call causes the link named `old` to be renamed as `new`. If `new` exists, it is first removed. [...]

The `rename()` system call guarantees that an instance of `new` will always exist, even if the system should crash in the middle of the operation.

Great, done deal! Wait ... Apple had a bug?

Apple's bug in `rename()` (from 2008 to 2011)

<http://www.weirdnet.nl/apple/rename.html>

Apple's implementation of `rename()` had an atomicity bug for more than 3 years (*it's `rename()` applied to symbolic links, to be precise*).

From `rename(2)`, on my Mac OS X Leopard (10.5.2) laptop :

The `rename()` system call guarantees that an instance of new will always exist, even if the system should crash in the middle of the operation.

Basically, what apple is saying here is that the `rename` system call is *atomic* : there should never be a time where new doesn't exist for the duration of the execution of the system call, and that after `rename` completes, the operation is either completed successfully or failed (eg. because the filesystem is read-only). For more details on how `rename()` is supposed to work, please read the SU specification. In this document I will empirically prove that `rename()` is not atomic in Mac OS X.

Demonstration was done with two concurrent processes (no crashes) →

dir/

└ A/

└ B/

└ link

└ templink



repeatedly do: create alternating “templink”, atomically rename it to “link”. Observation: “link” does not always exist